

8) API and Integraton

Last updated by | Mennell, Jacob | 19 May 2026 at 14:11 BST

API And Integration

Purpose

This page describes the API surface exposed by the solution and explains how client applications and other systems interact with the platform.

Scope

The API is the main synchronous integration boundary for the system. It exposes endpoints for:

- claims and claim summaries
- claim documents
- orchestration and job triggering
- model access and administration
- feedback capture
- selected computation utilities

The codebase shows the API implemented as a FastAPI application mounted under `/api/`.

API Structure

The application organizes routes into logical groups:

- `/api/claims`
- `/api/orchestration`
- `/api/models`
- `/api/documents`
- `/api/feedback`
- `/api/computation`
- `/api/health-check`

This structure keeps domain concerns separate while giving the frontend and other clients one consistent backend surface.

Authentication And Access Control

The API code and service documentation show a JWT bearer-token approach aligned to Azure Active Directory. Route groups are protected by role checks, with access enforced through dependencies in the FastAPI application.

The documented roles include application user, admin, and system-level access. This means the API is intended to support both end-user scenarios and privileged operational or service scenarios through the same core surface, with role-based controls deciding which actions are allowed.

Claims And Document Access

The claims routes provide claim-level retrieval and summary capabilities. Based on the route code and README, these include:

- streaming claim summaries for landing-page style views
- fetching a specific claim by claim ID
- fetching a claim by external business identifiers such as UMR and UCR
- assigning or updating claim ownership
- returning ordered document lists for a claim

The documents routes extend this by supporting document-specific actions such as:

- downloading a file by document ID
- rendering chunk-based content back into a PDF response when appropriate
- updating stored document classification metadata

This gives the API a dual role: expose claim-centric business data and expose underlying document assets in a usable form.

Orchestration Endpoints

The orchestration routes expose asynchronous workflow triggering. The route code shows support for:

- running jobs for explicitly supplied requests
- running jobs over recently discovered claims
- cleaning up stale jobs
- purging queues and marking corresponding work as revoked

This shows that the API is not only a data access surface; it is also a control surface for initiating and managing background processing.

Model And Access Administration

The models routes support administrative access patterns, including:

- listing models and their assigned users
- checking a user's model access
- granting model access to users
- deactivating model access

This supports a model-driven application pattern where user access is not global by default and can vary by claim model or operating context.

Feedback Capture

The feedback routes provide module-level feedback capture and retrieval tied to a claim and check name. The route code shows support for:

- retrieving the latest feedback entry
- patching the most recent feedback for a claim/check/user combination
- adding new module-level feedback entries

This is important because it gives the system a mechanism for capturing human feedback about AI-assisted outcomes without conflating that with the primary claim record.

Integration Patterns

The solution supports two main API integration patterns:

1. frontend-driven synchronous usage

The browser-based application calls API endpoints to fetch claim summaries, claim details, documents, extracted information, and check-related data.

2. system or operational triggering

Orchestration endpoints allow clients or operational flows to trigger asynchronous background work that is later reflected in stored state and surfaced back through the API.

Error And Response Behaviour

The service documentation shows JSON as the primary response format and documents common HTTP status patterns such as 401, 403, 404, 422, and 500. Certain endpoints use streaming responses, especially where large result sets or generated document responses are involved.

This is a pragmatic API design choice for a platform that mixes business entities, operational jobs, and reconstructed document content.

API Integration Diagram



flowchart LR

```
Client[Frontend or External Client] --> API[FastAPI API]
```

```
API --> Claims[Claims Routes]
```

```
API --> Docs[Documents Routes]
```

```
API --> Orch[Orchestration Routes]
```

```
API --> Models[Model Access Routes]
```

```
API --> Feedback[Feedback Routes]
```

```
API --> Compute[Computation Routes]
```

```
Orch --> Workers[Background Workers]
```

```
Claims --> DB[(Database)]
```

```
Docs --> DB
```

```
Feedback --> DB
```

```
Models --> DB
```

Design Decisions

- The API surface is grouped by domain rather than exposing one monolithic endpoint family.
- Role checks are centralized at route registration level, which keeps authorization intent explicit.

- Background workflow initiation is exposed through the same API boundary as data retrieval, making the platform easier to integrate with from a client perspective.
- Streaming responses are used where payload size or generated content makes that preferable to naïve in-memory JSON responses.

Operational Notes

- Orchestration endpoints trigger asynchronous work rather than returning a final business result inline.
- Document download behaviour varies by document type and ingestion state.
- Certain API behaviour depend on persisted ingestion or workflow outputs being available first.